

Week 6: 完成させて世界に 公開する

教材 — Week 6: 完成させて世界に公開する

このWeekのゴール: あなたのサービスが、誰でもアクセスできる URL で世界に公開される
所要時間: 動画 50 分 + 教材 90 分 + 宿題 30 分 **このWeekで身につく力:** 「自分のサービスを世界に届ける」一連の手順

-----|-----|-----|| **GitHub** | コードの保管庫 | 無料 || **Vercel** | 公開サーバー | 無料 (個人利用) || **ドメイン** (任意) | あなた専用 URL (例: my-name.com) | 年 1,000~2,000 円 |

レッスン 6-1: 「公開する」とは (5 分)

Web サイトが見える仕組み (3 分で理解)

```
あなたが index.html を作る
↓
Vercel というサーバーに「これを置いて」と頼む
↓
Vercel が URL を発行 (例: yamada.vercel.app)
↓
誰かがその URL にアクセス
↓
Vercel が index.html を返す
↓
ブラウザで表示 ✨
```

これが「Web」の本質です。「**ファイルを置いて、URL を発行してもらう**」だけ。

公開する前のチェック

公開前に、以下が出来ているか確認:

- ☐ サイトが PC で正しく動く
- ☐ スマホ表示も崩れない
- ☐ エラーが出ない
- ☐ AI 機能も動く

特に Week 4-5 で追加した機能が、**ローカルで完璧に動く** ことを確認してから次へ。

レッスン 6-2: Vercel で公開する（無料）（20 分）

ステップ全体像

- ① GitHub アカウントを作る（無料）
- ② プロジェクトを GitHub にアップロード（Claude にお任せ）
- ③ Vercel アカウントを作る（GitHub 連携で OK）
- ④ Vercel に GitHub プロジェクトを接続
- ⑤ 環境変数を設定（API キー等）
- ⑥ デプロイ → URL 発行 ✨

10～20 分で完了します。

① GitHub アカウントを作る

1. <https://github.com> にアクセス
2. 「Sign up」で新規登録（無料）
3. ユーザー名・メール・パスワードを設定
4. メール認証を完了

② プロジェクトを GitHub にアップロード

Claude Code に頼みます：

このプロジェクトを GitHub にアップロードしたい。
私の GitHub ユーザー名：yamada-taro

【やってほしいこと】

1. `.gitignore` ファイルを作成（`node_modules`, `.env.local` 等を除外）
2. `README.md` を整備
3. GitHub に新しいリポジトリを作る手順を教えて
4. ローカルから `push` する手順を教えて

→ Claude が手順を全部教えてくれます。

補足: `.env.local` は絶対に `push` しない

`.env.local` には API キーなどの **秘密情報** が入っています。これを GitHub に上げると **誰でも盗める状態** になります。`.gitignore` に `.env.local` を必ず含めること（Claude が自動でやってくれます）。

③ Vercel アカウントを作る

1. <https://vercel.com> にアクセス
2. 「Sign Up」をクリック

3. 「Continue with GitHub」を選択（GitHub アカウントで連携）
4. 個人利用 → Hobby プラン（無料）

④ Vercel に GitHub プロジェクトを接続

1. Vercel の Dashboard で「Add New... → Project」
2. 表示された GitHub リポジトリ一覧から **自分のプロジェクト** を選ぶ
3. 「Import」をクリック
4. プロジェクト設定画面が出る

⑤ 環境変数を設定

`.env.local` に書いた API キーを Vercel にも設定する必要があります。

Vercel のプロジェクト設定画面で:

1. 「Environment Variables」セクション
2. **Key** と **Value** に、`.env.local` の内容を 1 つずつコピー:

```
NEXT_PUBLIC_SUPABASE_URL = https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY = eyJhbG...
SUPABASE_SERVICE_ROLE_KEY = eyJhbG...
ANTHROPIC_API_KEY = sk-ant-...
```

3. 「Production」「Preview」「Development」全てにチェック

⑥ Deploy

1. 一番下の「Deploy」ボタンをクリック
2. 1〜3 分待つ
3. 「Congratulations!」画面 + 自分の URL（例: `https://yamada-portfolio-xxx.vercel.app`）

→ **その URL をブラウザで開く** → あなたのサイトが世界中からアクセス可能に 🚀

動作確認

1. URL をスマホで開く → サイトが表示される
2. 家族・友達に URL を送る → 「ちゃんと見えるよ」と言われる
3. AI 機能、ログイン機能も動くか確認

レッスン 6-3: 自分のドメインをつける (15 分)

「.vercel.app」が嫌なら、自分専用 URL を取得

`yamada-portfolio.vercel.app` ↑ これは無料だけど Vercel の URL。自分専用になりたいなら、ドメイン（独自 URL）を取得します。

ドメイン取得 (年 1,000～2,000 円)

おすすめドメイン取得サービス

サービス	おすすめ度	価格目安
お名前.com	☆☆☆	.com で 1,500 円/年～
Cloudflare Registrar	☆☆☆☆☆	卸値で安い、信頼性高
Google Domains	(現在停止中)	—
ムームードメイン	☆☆☆	価格安い、UI 日本語

初心者は **Cloudflare Registrar 推奨**: 安い + 設定簡単 + 安心。

ドメイン名の決め方

✓ 良い例

- yamada-taro.com (自分の名前)
- ai-counselor.app (サービス名)
- 5min-recipe.com (覚えやすい)

✗ 微妙な例

- my-super-cool-amazing-website.com (長すぎ)
- 12345.xyz (怪しげ)
- my_website_2026.com (_ や数字は避ける)

Vercel に独自ドメインを接続

1. Vercel プロジェクトの「Settings → Domains」
2. 「Add Domain」で取得したドメインを入力
3. 表示される **DNS 設定** をドメイン会社で設定
4. 5～30 分待つ → ドメインが繋がる ✨

困ったら Claude に:

取得したドメイン `yamada.com` を Vercel に接続したい。
お名前.com の DNS 設定で何をすればいい?

レッスン 6-4: 最終チェックリスト (15 分)

公開前に確認したい 10 項目

サイト品質 (5)

- ☐ PC で全ページが正しく表示
- ☐ スマホで全ページが正しく表示
- ☐ 全ページのリンクが切れていない
- ☐ フォント・配色が統一されている
- ☐ 画像が崩れていない・読み込まれる

機能動作 (3)

- ☐ お問い合わせフォームが動く (実際に送信して試す)
- ☐ ログイン・会員登録が動く
- ☐ AI 機能が動く (Vercel 側でも API キー設定済みか)

SEO・SNS シェア (2)

- ☐ 各ページにタイトル・説明文が設定されている
- ☐ OGP 画像 (SNS シェア時のサムネイル) が表示される

Claude に最終チェックを頼む

公開前の最終チェックをお願い。

【確認してほしい項目】

1. 全ページのタイトル・description は適切か
2. OGP 画像の設定は完了しているか
3. ファビコンは設定されているか
4. robots.txt と sitemap.xml は設定されているか
5. 404 ページは作られているか
6. 環境変数が必要なものは全て `.env.local` に列挙されているか

問題があれば修正して。

→ Claude が一括チェック・修正してくれます。

OGP 画像とは

SNS (X、Facebook 等) で URL をシェアした時に表示されるサムネイル画像。

OGP 画像を作って。
1200x630px のサイズで。
中央に「サイト名」を大きく表示。
背景は紺×ゴールドで高級感のあるデザインで。

→ Claude が自動生成 or デザイン案を提案します。（より凝りたいなら Canva で作って画像を `public/og-image.png` に置く）

レッスン 6-5: 友達・家族に使ってもらう（10 分）

URL を送って試してもらう

URL を送るときのテンプレ:

〇〇というサイトを作ったよ。試しに見て、感想を聞かせてくれない?
URL: `https://〇〇`

特に知りたいこと:

- 見た目はどう?
- スマホで見ても綺麗?
- 何か変なところある?

フィードバックの聞き方

「率直な意見が欲しい」と最初に言うと、本音が聞けます。

良いフィードバック例:

- 「ヘッダーの文字が読みにくい」
- 「お問い合わせフォーム、送ったらどうなるか分からなくて不安」
- 「スマホでメニューが押しにくい」

これらを Claude に頼んで修正:

家族から「ヘッダーの文字が読みにくい」と言われた。
コントラストを上げて、文字をもう少し大きくして。

→ 5 分で修正完了。再デプロイされる（GitHub に push すれば Vercel が自動で再デプロイ）。

この体験が、最も成長する瞬間

「自分が作ったサービスを、他人が使ってくれた」

この感覚が、Web 個人開発の **最大の喜び** であり、**最大の学び** です。

恥ずかしがらずに、まずは家族・親友 1 人に見せてください。

レッスン 6-6: 実習: 本番デプロイ (25 分)

進め方

```
Step 1: Claude に GitHub 用の準備を頼む (5 分)
    ↓
Step 2: GitHub にプロジェクトを push (5 分)
    ↓
Step 3: Vercel にプロジェクトをインポート (5 分)
    ↓
Step 4: 環境変数を設定 (5 分)
    ↓
Step 5: Deploy → URL 発行 ✨
    ↓
Step 6: スマホ・友人で動作確認 (5 分)
    ↓
Step 7: コミュニティに URL 投稿
```

「自動再デプロイ」の仕組み

Vercel に GitHub を接続したら、**それ以降はコードを更新するたびに自動で再デプロイ** されます。

```
Claude にコード修正を頼む
    ↓
ローカルで動作確認
    ↓
git add . && git commit -m "〇〇を修正"
git push
    ↓
Vercel が自動で気づいて、1~2 分後に本番反映 ✨
```

これも面倒なら、Claude に：

```
git に変更を反映して push して
```

→ Claude が全部やってくれます。

GitHub 関連（5 フレーズ）

こう言えば...	こうなる
「.gitignore を作って。node_modules、.env.local を除外」	必須ファイル除外
「このプロジェクトを GitHub にアップする手順を教えて」	手順書をくれる
「git に変更を push して」	全自動で push してくれる
「README.md を整備して」	説明文書を書いてくれる
「コミットメッセージを良い感じで生成して push して」	コミット内容に沿ったメッセージで

Vercel 公開（5 フレーズ）

こう言えば...	こうなる
「Vercel にデプロイする手順を教えて」	手順書をくれる
「環境変数の一覧を教えて」	必要な ENV を列挙
「ローカルでは動くのに本番で動かない。原因調査して」	デプロイ問題のデバッグ
「ビルドエラーを直して（エラー文を貼る）」	デプロイ時のエラー解決
「カスタムドメインを設定する手順を教えて」	DNS 設定手順

SEO・OGP（4 フレーズ）

こう言えば...	こうなる
「全ページの SEO 設定（title, description）をして」	meta タグ整備
「OGP 画像を設定して」	SNS シェア用画像
「sitemap.xml と robots.txt を作って」	検索エンジン対応
「ファビコンを設定して」	ブラウザタブのアイコン

その他（4 フレーズ）

こう言えば...	こうなる
「404 ページをデザインを揃えて作って」	存在しない URL の対応
「Google Analytics を入れて」	アクセス解析
「Vercel Analytics を入れて」	より詳細な解析
「読み込み速度を改善して」	パフォーマンス最適化

Vercel 公開手順書（完全版）

手順 1: GitHub の準備

```
# プロジェクトフォルダで（ターミナルから）
cd ~/my-apps/my-website

# Git 初期化（既に初期化済みならスキップ）
git init

# .gitignore を確認（.env.local が含まれているか）
cat .gitignore | grep .env

# 全ファイルをステージング
git add .

# コミット
git commit -m "Initial commit"
```

手順 2: GitHub にリポジトリを作る

- 1. GitHub にログイン
- 2. 右上の「+」 → 「New repository」
- 3. **Repository name:** my-website 等
- 4. **Public** か **Private** を選択（Private 推奨）
- 5. README は **追加しない**（既にあるため）
- 6. 「Create repository」をクリック
- 7. 表示されるコマンドをターミナルにコピペ:

```
git remote add origin https://github.com/your-username/my-website.git
git branch -M main
git push -u origin main
```

手順 3: Vercel に接続

1. Vercel にログイン
2. Dashboard で「Add New」→「Project」
3. GitHub の自分のアカウントを認証
4. リポジトリ一覧から `my-website` を選択 → 「Import」

手順 4: 環境変数の設定

「Environment Variables」セクションで、以下を全て追加（`.env.local` の内容と同じ）：

```
NEXT_PUBLIC_SUPABASE_URL = https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY = eyJhbG...
SUPABASE_SERVICE_ROLE_KEY = eyJhbG...
ANTHROPIC_API_KEY = sk-ant-...
NEXT_PUBLIC_ADMIN_EMAIL = your@email.com
（必要に応じて他も）
```

各変数で **Production / Preview / Development** の 3 つにチェック。

手順 5: Deploy

1. 「Deploy」ボタンをクリック
2. 1〜3 分待つ
3. 「Congratulations!」画面が出る
4. 「Visit」ボタンで自分のサイトを開く 🎉

手順 6: その後の更新

```
# コードを修正
# ↓
git add .
git commit -m "〇〇を修正"
git push
# ↓
# Vercel が自動で再デプロイ（1〜2 分）
```

または Claude に「git に変更を push して」と頼むだけ。

公開前チェックリスト 10 項目

- ☐ PC で全ページが正しく表示
- ☐ スマホで全ページが正しく表示

- ☐ 全ページのリンクが切れていない
 - ☐ お問い合わせフォームの動作確認
 - ☐ ログイン・会員登録の動作確認
 - ☐ AI 機能の動作確認
 - ☐ 各ページのタイトル・description 設定済み
 - ☐ OGP 画像が設定されている（SNS シェアテスト）
 - ☐ ファビコンが設定されている
 - ☐ 404 エラーページが作られている
-

ドメイン取得ガイド

推奨サービス比較

Cloudflare Registrar（一番おすすめ）

- **価格:** 卸値で最安（.com は約 \$10/年）
- **更新料金:** 値上げなし
- **DNS:** 同じく Cloudflare で管理可能（高速・無料）
- **欠点:** 英語 UI、日本円決済不可（クレジットカード）

お名前.com（日本人に人気）

- **価格:** 初年度安い（300 円～）が **更新時に高くなる**（3,000 円超）
- **DNS:** 自社 DNS 利用可
- **メリット:** 日本語、初心者向け
- **欠点:** 営業メールが多い、UI が古い

ムームードメイン

- **価格:** 1,500 円/年～
- **メリット:** シンプル、日本語、安定
- **欠点:** 機能少なめ

ドメインの選び方

- ✓ できれば .com or .jp (信頼度高)
- ✓ 短く (10 文字以内が理想)
- ✓ 覚えやすい
- ✓ サービス名と一致 or 自分の名前
- ✓ ハイフンや数字は避ける (覚えにくい)
- ✗ 流行りの拡張子 (.xyz, .info) は避ける
- ✗ ブランド名と紛らわしい (appleの.com 等)
- ✗ 商標登録されている名前

Week 6 チェックリスト

- ☐ GitHub アカウントを作った
- ☐ プロジェクトを GitHub に push した
- ☐ Vercel アカウントを作った (GitHub 連携)
- ☐ Vercel にプロジェクトを Import した
- ☐ 環境変数を Vercel に設定した
- ☐ Deploy が成功し、URL が発行された
- ☐ PC で URL を開いて動作確認
- ☐ スマホで URL を開いて動作確認
- ☐ 家族 or 友人に URL を送って見てもらった
- ☐ フィードバックを反映した
- ☐ OGP 画像・ファビコン・SEO 設定完了
- ☐ コミュニティに URL を投稿 (晴れ舞台!)
- ☐ (任意) 独自ドメインを取得・設定した

トラブルシューティング

Q1. Deploy したらビルドエラーが出ました

A: エラー画面のログを Claude に渡す:

Vercel デプロイで以下のエラーが出た。直して:
[エラー文をコピペ]

→ ほとんどは「環境変数が設定されていない」「ローカルにしかないファイルを参照している」等。

Q2. ローカルでは動くのに、本番で AI 機能が動かない

A: 環境変数の設定漏れ。Vercel の「Environment Variables」を確認:

- 全ての `.env.local` の値が入っているか
- 「Production」にもチェックが入っているか
- 値の前後に余分なスペースがないか

設定後、再デプロイ（Vercel の「Redeploy」ボタン）。

Q3. 独自ドメインが繋がらない

A: DNS の伝播に時間がかかります（最大 48 時間、通常は 30 分～2 時間）。

- Vercel の Domains 画面で「Valid Configuration」になっているか確認
- ならない場合は、ドメイン会社の DNS 設定が間違っている可能性
- Claude に DNS 設定を確認してもらう

Q4. URL を友達に送ったら「アクセスできない」と言われた

A:

- URL のタイポを確認
- HTTPS（`https://`）になっているか確認
- 自分のスマホで別の Wi-Fi（携帯回線）から開いて確認

Q5. Supabase の RLS（Row Level Security）でエラーが出る

A: 本番ではセキュリティが厳しくなります。Claude に頼む:

Supabase の RLS 設定を本番用にして。
○○テーブルは認証済みユーザーが自分のデータだけ読み書き可能、
管理者は全て可能、にして。

Q6. `.env.local` の内容が GitHub に push されてしまった!

A: 緊急対応:

1. **すぐ全ての API キーを再発行**（漏れたキーは無効化）
2. `.gitignore` に `.env.local` を追加
3. 「`git rm --cached .env.local`」でリポジトリから削除
4. push し直す

今週の宿題（必須）

提出物 1: 公開 URL のシェア

コミュニティの「**Week 6 完了報告**」スレッドに投稿:

🎉 【Week 6 完了報告】 公開しました！

【公開 URL】

<https://〇〇.vercel.app>

（または独自ドメインの場合はそちら）

【サービス概要】

〇〇な人のための、〇〇できるサービスです

【使い方】

1. 〇〇
2. 〇〇

【特に試してほしい機能】

〇〇

【Vercel デプロイで詰まったポイント】

〇〇

【感動した瞬間】

〇〇

提出物 2: 友達 / 家族からのフィードバック

最低 1 人に URL を送ってフィードバックをもらい、コミュニティに共有:

【もらったフィードバック】

- 〇〇
- 〇〇

【それを受けて修正したこと】

- 〇〇

【まだ反映できていないが、改善したいこと】

- 〇〇

コラム: 「URL を持つ」という新しい体験

私が初めて自分のサービスを公開した日のことを覚えています。

<http://example.vercel.app> を妻に送って「これ、見て」と言ったら、妻のスマホに表示された。それまで「自分の頭の中・PCの中」にしか無かったものが、**世界中からアクセスできる場所**に存在するようになった。

この瞬間、世界の見え方が変わりました。

「自分は、世界に何かを届けられる人間だ」

これが、Web 個人開発の最も尊い体験だと思います。

Week 6 を終えたあなたは、もう **「サービスを世界に届けられる人」** です。おめでとう、本当に。

次の Week では、これに **「お金を受け取る仕組み」** を入れて、**ビジネス** にしていきます。

次の Week へ

Week 6 おつかれさまでした！あなたのサービスは世界に公開されました。次は **「ビジネスとして成立させる」** 段階。決済を入れて、お金を受け取れる状態にします。

→ [Week07-お金を受け取れるようにする.md](#)